

DexVision / Hand2Bot

Simulator-first dexterous manipulation, scripted skill data, compact policy learning, and future language-guided orchestration

CURRENT POSITION: Levels 1-3 and Level 4.4 are complete. The active pipeline uses deterministic scripted experts, not live hand control. Level 4.5A next collects the pick/place anchor. Level 4.5B then expands procedural diversity and tests data scaling before the visual export, audit, and immutable release.

This overview is generated from docs/project_overview.md. CURRENT_STATUS.md remains the checkpoint source of truth.

Project identity

DexVision is a simulator-first dexterous manipulation and robot-learning project. Deterministic experts control a Shadow Hand in MuJoCo, successful and failed trajectories are recorded with full safety provenance, and compact PyTorch policies learn bounded reusable skills. Rendered perception later maps images into the same typed world-state interface.

The eventual language model is a planner, not the robot controller. It turns a request into typed calls to qualified skills. A deterministic supervisor checks parameters, world state, safety, timeouts, and terminal success.

The repository retains an earlier camera/hand-tracking prototype and its Level 2 dataset as historical work. The Level 4+ system does not use live hand control for demonstrations, corrections, training, or execution.

Current status

Levels 1 through 3 are complete. Level 3 proved that loading, training, checkpoint selection, and frozen rollout evaluation are reproducible, but no policy trained on the narrow Level 2 release passed its closed-loop gates.

Level 4.3 then replaced the unreliable human-control assumption with deterministic simulator-state experts. Reach, button press, constrained push, grasp-and-lift, and complete pick/place experts now regenerate, recompute, and replay successfully. Small state-only button and push learning probes also passed their held-out gates without larger models, images, or action chunking.

Level 4.4 is complete with 60 scripted core episodes: 20 reach, 20 push, and 20 button trajectories across all required cells. Level 4 is active at checkpoint 4.5A, which collects the frozen complete pick/place anchor matrix. That 114-episode anchor validates coverage, provenance, replay, and segmentation; it is not the final learning-data claim. Level 4.5B then expands every nominal coverage cell with independently seeded continuous variation, and Level 4.6 does the same for failures/corrections. The release-candidate floor is 1,112 accepted episodes, with a validation-only scaling gate that can require more. The remaining work also adds rendered visual annotations, dataset audit, and an immutable release.

Architecture

```
typed task + simulator world state
  -> deterministic expert + copied-state safety validation
  -> MuJoCo execution
  -> requested / commanded / applied actions + causal phases
  -> replay / quality / terminal recomputation
  -> versioned dataset and frozen splits
  -> compact learned skill policy
  -> rendered detector / tracker / pose
  -> supervised typed skill executor
```

```
future Level 7:
language request -> typed plan -> deterministic supervisor -> qualified skills
```

The initial learned interface is deliberately small. State-grounded policies receive task-relative geometry, causal phase, relevant robot state, and previous applied action. They emit bounded task-local deltas or residuals around the deterministic expert. More expressive temporal models are allowed only after a measured failure justifies them.

Roadmap

Level 1 — Historical hand-tracking prototype (complete)

OpenCV and MediaPipe track a hand and drive the MuJoCo Shadow Hand. This remains a reproducible prototype, not an active manipulation-data source.

Level 2 — Demonstrations and data infrastructure (complete)

The system records, replays, validates, relabels, filters, summarizes, and benchmarks demonstrations. The immutable legacy release contains 55 reach, 55 button, and 101 push successes with manifests and checksums. These episodes are seed evidence only and do not satisfy Level 4 coverage.

Level 3 — Learning feasibility (complete)

Level 3 built the PyTorch loader, small goal-conditioned baseline, reproducible training loop, validation-only checkpoint selection, frozen MuJoCo evaluation, and action/data diagnostics. Its negative qualification result motivated the new expert, action, phase, and dataset contracts.

Level 4 — Comprehensive scripted skill dataset (active)

Level 4 uses deterministic experts, simulator truth, complete action/safety records, causal phases, whole-session split ownership, held-out objects/goals, and aligned rendered visual labels. Its first 114 episodes are a frozen integration/coverage anchor. The learning release must contain at least 992 nominal expert episodes and 120 separately labeled failure/correction episodes, for 1,112 accepted episodes total, with unique resets and coverage-owned variation. This is a floor rather than proof of sufficiency: validation-only scaling evidence may require another versioned tranche. No live-control episode is required or included in the active release.

Required skills are:

```
reach_object
pick_object
place_held_object
push_object_to_target
press_button
```

The optional dial remains deferred. Level 4 ends with coverage and bias reports, frozen train/validation/test manifests, checksums, and an immutable release.

Level 5 — Full-scale skill learning and qualification (planned)

Level 5 trains the smallest justified state-grounded skill policies first. It then qualifies conventional rendered detector/tracker/pose perception, measures perception-grounded rollouts, and admits only compatible qualified policies to a typed supervised executor. Training consumes the immutable scripted release; it does not recollect human demonstrations to rescue failures.

The deterministic expert remains a required baseline. A residual policy that does not materially beat its zero-residual ablation is not presented as a standalone learned skill, even if the combined executor succeeds.

Level 6 — Robustness and portfolio polish (planned)

Level 6 packages measured results, architecture diagrams, deterministic expert and policy demonstrations, reproducibility/CI, cross-platform commands, troubleshooting, and optional dataset export. The historical hand-tracking prototype is clearly separated from the active system claim.

Level 7 — Language-guided orchestration (future)

An LLM or deterministic planner consumes symbolic world state and versioned skill cards. It proposes typed plans; the supervisor validates and executes them. Language never emits high-rate actuator commands.

Tabletop workcell pilots

Level 5 validates three related but materially different pilots:

1. Workspace clearing: return loose rigid parts to bins, using push where useful.
2. Inspection-station operation: place a part on the inspection pad and press Start.
3. Workspace setup: arrange components at marked positions for the next job.

A combined work order exercises several skills without expanding into kitchen tasks, cutting, pouring, deformables, arbitrary-object grasping, or open-world robotics.

Dataset release policy

Working data stays under ignored `data/demos/`. Immutable releases use Git LFS when they fit the documented threshold and always include manifests, split files, retrieval instructions, and SHA-256 checksums. Existing archives and accepted episodes are never overwritten.

Honest project claim

DexVision currently demonstrates deterministic expert generation, replayable and safety-audited workcell data, reproducible behavior cloning, and frozen closed-loop evaluation. Level 4 will determine whether the full scripted workcell release is complete and balanced. Level 5 will determine which compact skills qualify under state and rendered-perception evaluation. Language-guided multi-skill behavior remains future work until those gates pass.